

```
data['Max Humidity'].mean()

90.027322404371589

data['Max Humidity'].sum()

32950

data['Max Humidity'].std()

9.10843757197798
```



Note: Most of the Pandas functions ignore NaNs, by default. These regularly occur in data and a convenient way of handling them must be established. This topic is covered more in detail later in this article.

The *std* and *sum* function can be used in a similar manner. Also, rather than running these functions on individual columns, you can run them on the entire dataframe, as follows:

```
data.mean()

Max TemperatureF      66.803279
Mean TemperatureF     55.683060
Min TemperatureF      44.181093
Max Dew PointF        49.549180
MeanDew PointF       44.057377
Min DewpointF        37.980874
Max Humidity          90.027322
Mean Humidity        67.860656
Min Humidity         45.193989
Max Sea Level PressureIn 30.108907
Mean Sea Level PressureIn 30.022705
Min Sea Level PressureIn 29.936831
Max VisibilityMiles    9.994536
Mean VisibilityMiles   8.732240
Min VisibilityMiles    5.797814
Max Wind SpeedMPH     16.418033
Mean Wind SpeedMPH    6.057377
Max Gust SpeedMPH     22.764384
CloudCover            2.885246
WindDirDegrees       189.704918
```

Using *apply* for bulk operations

As we have already seen, functions like *mean*, *std* and *sum* work on entire columns, but sometimes it may be useful to apply our own functions to entire columns of the dataframe. For this purpose, Pandas provides the *apply* function, which takes an anonymous function as a parameter and applies to every element in the column. In this example, let us try to get the square of every element in a column. We can do this with the following code:

```
data['Max Humidity'].apply(lambda d: d**2)

0      5476
1      6084
2      8100
3      8649
4      8649
5      8100
...
...
...
361    8464
362    7225
363    7744
364    5625
365    2916
Name: Max Humidity, Length: 366
```



Note: In the Lambda function, the parameter *d* is implicitly passed to it by Pandas, and contains each element of the *a* column.

Now you may wonder why you can't just do this with a loop. Well, the answer is that this operation was written in one single line, which saves code writing time and is much easier to read.

Dealing with NaN values

Pandas provides a function called *isnull*, which returns a 'True' or 'False' value depending on whether the value of an element in the column is NaN or None. These values are treated as missing values from the dataset, and so it is always convenient to deal with them separately. We can use the *apply* function to test every element in a column to see if any NaNs are present. You can use the following command:

```
e = data[' Events'].apply(lambda d: pandas.isnull(d))
e
0      True
1     False
2     False
3      True
4      True
5     False
...
361    False
362     True
363     True
364     True
365     True
Name: Events, Length: 366
```